

MicroGridsPy - Installation & Launch Guide

This guide explains **how to install** (only once) and **then run** (every time you need it) the MicroGridsPy Streamlit application on your computer. The tool **runs offline**, directly on your computer, without needing internet once installed.

Installation (DO THIS ONLY ONCE)

You will do this part only when installing the application for the first time.

Step 1 - Download the App folder

1. Open the GitHub link: <https://github.com/MicroGridsPy/MicroGridsPy>
2. Click Code (green button)
3. Click Download ZIP
4. Unzip the folder in a location you can remember

Take note of the path, you will use it later (Example: `C:\Users\onori\Desktop\MicroGridsPy`).

GitHub is an online platform where developers publish and maintain software projects. The MicroGridsPy application is stored there and publicly accessible.

By downloading the ZIP file, you are simply obtaining the folder containing the application code, configuration files, and all resources needed to run it on your computer.

Once downloaded, the folder behaves like any regular folder on your machine, you don't need Git or any special tools to use it.

Step 2 - Create a dedicated environment

1. Open Anaconda Prompt
2. Change directory to your MicroGridsPy folder using: `cd "path/to/folder"`. Example: `cd "C:\Users\onori\Desktop\MicroGridsPy"`
3. Run the installation command: `conda env create -f environment.yml`

This will:

- ✓ create the `mgpy_planning` environment
- ✓ install Python
- ✓ install all required packages (including streamlit)

If everything concludes without errors, the installation is complete.

Launching the App (EVERY TIME YOU WANT TO USE IT)

Once the MicroGridsPy application has been installed successfully for the first time, running it again is simple and does not require repeating the installation process. From now on, every session consists of three short steps: activate the environment, point to the app folder, and launch the Streamlit interface.

The logic behind these steps is the following:

1. Activating the environment ensures your computer uses the correct Python version and libraries associated with MicroGridsPy.

2. Navigating to the app folder tells the system where the application code lives.
3. Running Streamlit launches the browser interface and displays the app.

Below are the detailed instructions.

Step 1 - Activate the Environment

Before launching the app, you must enable the environment created during installation:

```
conda activate mgpy_planning
```

This command switches your terminal into the MicroGridsPy “workspace”. You should see the environment name appear at the beginning of the terminal line (e.g., (mgpy_planning)), indicating that it is active.

Step 2 - Go to the App Folder

Next, tell the terminal where the application files are located:

```
cd "path/to/folder" (Example: cd "C:\Users\onori\Desktop\MicroGridsPy")
```

Once inside the folder, the terminal can correctly locate and execute the application’s entry point.

Step 3 - Run the Streamlit Application

Now you can start the application:

```
streamlit run Home.py
```

After a few seconds, your default web browser (Chrome, Firefox, Edge, etc.) should open automatically and display the MicroGridsPy interface. The application runs locally, so the browser is simply acting as a user-friendly window for interacting with the tool.

Welcome to MicroGridsPy!

MicroGridsPy Planning is the techno-economic optimization layer of the MicroGridsPy ecosystem.

It supports off-grid and weak-grid mini-grid design, combining renewable generation, batteries, generators, optional grid interaction, stochastic scenarios, and both typical-year and multi-year capacity-expansion formulations. The broader ecosystem connects resource assessment, demand modelling, distribution design, planning, and detailed operational analysis into one coherent workflow.



Core planning engine

MicroGridsPy Planning

Techno-economic optimization of mini-grid systems under deterministic or stochastic assumptions. Use it to size renewables, batteries, generators, and grid interaction with either a representative typical year or a multi-year dynamic formulation with capacity expansion.

[GitHub repository](#)

Open Project Setup

What to Expect When Launching

When launching the app, you may encounter a few harmless prompts:

- Streamlit may ask for an email → simply press Enter to skip.
- Windows may ask for permission to run Python scripts → choose Yes.

- A browser tab will open automatically. If you close it, you can reopen the app by typing localhost:8501 into the browser address bar (as long as the terminal remains running).

To stop the application, simply close the terminal window, or press CTRL + C in the terminal

About Future Updates

After installation, two components coexist on your machine:

- The environment (mgpy_planning) → contains Python + all required packages
- The MicroGridsPy project folder → contains the actual application code

These two components are intentionally separated. The environment rarely changes, whereas the application code may be updated more frequently.

How to Update the Application Code

If a new version of the MicroGridsPy application becomes available on GitHub:

- you do not need to reinstall or recreate the environment
- you do not need to re-download packages or dependencies

Instead, simply:

1. Download the updated folder from GitHub (ZIP as before)
2. Replace your old MicroGridsPy folder with the new one
3. Continue using your existing mgpy_planning environment

This workflow ensures smooth updates without repeating the installation burden.

When Would You Recreate the Environment?

Recreating the environment is rarely needed and should only be done in specific cases, such as:

- new dependencies added (e.g., new Python packages required)
- change of Python version (e.g., moving from Python 3.10 to 3.12)
- explicit instructions in an update note or release announcement
- corrupted or partially removed environment

In these cases, you can remove the old environment:

```
conda remove -n mgpy_planning --all
```

and then recreate it using:

```
conda env create -f environment.yml (e.g., new dependencies added).
```